

CHƯƠNG 4: TRIỂN KHAI HỆ THỐNG VÀ ĐÁNH GIÁ THỰC NGHIỆM

4.1. Môi trường và cấu hình thực nghiệm

4.1.1. Cấu hình phần cứng tại nút biên

Hệ thống được thử nghiệm trên một máy chủ độc lập đóng vai trò là trung tâm dữ liệu thu nhỏ, mô phỏng lại môi trường trạm xử lý biên của lực lượng cảnh sát giao thông tại các nút giao thông lớn. Việc lựa chọn cấu hình được tính toán kỹ lưỡng dựa trên bài toán điểm nghẽn dữ liệu giữa bộ xử lý trung tâm CPU và bộ xử lý đồ họa GPU.

Bảng 4.1. Cấu hình phần cứng máy tính thực nghiệm

Linh kiện	Thông số kỹ thuật chi tiết	Vai trò trong hệ thống SmartTraffic
CPU	Intel Core i5-12500H - 12 Cores (4 P-Cores, 8 E-Cores) - 16 Threads, Xung nhịp cơ bản 2.50 GHz	Đảm nhiệm xử lý luồng sự kiện bất động bộ của FastAPI, vận hành thuật toán tìm kiếm Vector FAISS CPU, và tiền xử lý khung hình Video H.264 trước khi đẩy vào GPU.
GPU	NVIDIA GeForce RTX 3050 Ti Laptop GPU 4 GB VRAM Hỗ trợ CUDA & Tensor Cores	Phân hệ tính toán và suy luận AI trung tâm đảm nhiệm quá trình nội suy cho mô hình YOLOv8 và bộ giải mã quang học OCR. Với 4 GB VRAM, hệ thống đóng vai trò như một Trạm xử lý biên đặt tại ngã tư.
RAM	16 GB (Tốc độ 3200 MT/s)	Cung cấp bộ nhớ để lưu trữ bộ đệm Redis, duy trì kho In-memory Vector Database (FAISS), và bộ nhớ đệm khung hình Video.
Lưu trữ	512 GB SSD NVMe	Đảm bảo tốc độ truy xuất siêu tốc khi nạp trong số mạng nơ-ron và lưu trữ tạm thời các ảnh cắt bằng chứng.

Phân tích về giới hạn phần cứng: cấu hình thực nghiệm mô phỏng chính xác một trạm xử lý biên được lắp đặt trực tiếp tại các tủ điều khiển ngã tư đường phố, thay vì sử

dung máy chủ đám mây chi phí cao. Thách thức kỹ thuật lớn nhất ở cấu hình này nằm ở dung lượng VRAM hạn chế của card đồ họa RTX 3050 Ti.

Khác với các ứng dụng sinh tạo chỉ chạy theo truy vấn rời rạc, mô hình YOLOv8 và bộ giải mã OCR phải quét liên tục từng khung hình của camera theo thời gian thực. Nếu nạp toàn bộ mô hình YOLO nguyên bản vào bộ nhớ 4 GB, hệ thống sẽ lập tức gặp lỗi tràn bộ nhớ VRAM. Để giải quyết rào cản tài nguyên này mà vẫn đảm bảo tốc độ xử lý thời gian thực, hệ thống áp dụng kỹ thuật tối ưu hóa mô hình thông qua lượng tử hóa trọng số và cố định kích thước lô xử lý BatchSize = 1. Cách tiếp cận này giúp chứng minh tính khả thi khi triển khai thương mại hóa hệ thống trong thực tế với chi phí phần cứng tối ưu.

4.1.2. Nền tảng phần mềm và thư viện sử dụng

Nhằm khai thác triệt để hiệu năng của cấu hình phần cứng tại trạm xử lý biên, việc lựa chọn và tổ chức ngăn xếp công nghệ phần mềm được thực hiện dựa trên các nguyên tắc: tối ưu hóa tài nguyên phần cứng giới hạn, triệt tiêu độ trễ truyền thông, đảm bảo tính bất đồng bộ hoàn toàn và khả năng đóng gói vi dịch vụ linh hoạt.

Bảng 4.2. Công nghệ phần mềm sử dụng trong hệ thống

Lớp kiến trúc	Công nghệ / Thư viện	Phiên bản	Chức năng và cơ chế kỹ thuật chuyên sâu
Môi trường cơ sở & Ảo hóa	Windows 11 & WSL2 docker	25H2 & v24.0.5	Vận hành môi trường nhân Linux (WSL2) tích hợp trực tiếp driver NVIDIA CUDA qua giao tiếp NVIDIA Container Toolkit. Giúp đóng gói ứng dụng vào container Docker, đảm bảo tính nhất quán giữa môi trường phát triển và triển khai thực tế.
Nền tảng thực thi lỗi	Python	3.10.9	Ngôn ngữ lập trình chính cho toàn bộ backend. Tích hợp thư viện uvloop (được viết bằng Cython dựa trên libuv của Node.js) để thay thế vòng lặp sự kiện (event loop) mặc định của